Here's a clear, professional explanation of your `audit.py` file. You can share this directly with architects or technical stakeholders.

Purpose of the Audit File

The `run_pre_clean_audit()` function performs a **pre-cleaning signal integrity audit**. It analyzes the raw dataset (using the ydata-profiling `report.json`) before any cleaning or feature engineering happens.

Its main goals are:

- Detect obviously useless columns (e.g., IDs, unique identifiers).
- Assess how much predictive signal exists in the features.
- Dynamically recommend safe thresholds for **PPS (Predictive Power Score)** and **VIF (Variance Inflation Factor)**.
- Identify weak features that are unlikely to help prediction.
- Provide a traceable JSON output (`pre_clean_audit.json`) for auditability and reproducibility in an MLOps pipeline.

This is a **smart, data-driven gate** that prevents over-cleaning or under-cleaning, especially important for sensitive domains like medical data (e.g., breast cancer diagnosis).

Line-by-Line Explanation + Architectural Decisions

```python
# 1. Dimensionality Context
n_vars = report['table']['n_var']
n_rows = report['table']['n']
```

Why?

The number of features (`n_vars`) and rows tells us the dataset's complexity. High-dimensional medical datasets (30–100+ features) behave differently from simple ones (Iris with 4 features). This context drives all later decisions.

```python
```

```
# 2. Identify Identity Columns
id_cols = [col for col, s in report['variables'].items()
           if s.get('is_unique') or s.get('n_distinct') == n_rows]
```

Why?

Columns that are unique per row (like `id`, `patient_id`, or index) have zero predictive value and can leak information or cause overfitting. Removing them early is a best practice. We use the profiling report because it already computed `is_unique` and `n_distinct` efficiently.

```
Python

# 3. Initial Dynamic Strategy
if n_vars > 50:
    pps_threshold, vif_threshold = 0.05, 5.0
elif n_vars < 15:
    pps_threshold, vif_threshold = 0.005, 20.0
else:
    pps_threshold, vif_threshold = 0.02, 10.0
```

Why these specific numbers? (Architectural Reasoning)

- **PPS Threshold**: Controls how aggressively we drop "weak" features (low predictive power).
  - High-dimensional data (>50 features) → higher threshold (0.05) because many features are expected to be weak/noisy.
  - Small data (<15 features) → very low threshold (0.005) because we cannot afford to drop anything useful.
  - Default (most cases) → 0.02 is a balanced, conservative value used in many production pipelines.

- **VIF Threshold**: Controls multicollinearity (when features are highly correlated with each other).
    - Strict VIF=5.0 for very high-dimensional data (aggressive removal of redundant features).
    - Lenient VIF=20.0 for small/simple datasets (preserve information).
    - Classic default is 10.0.

This dynamic approach makes the pipeline **adaptive** instead of using one fixed threshold for all datasets.

```python
# 4. Preliminary Deepchecks Audit
audit_df = df.drop(columns=id_cols)
cat_features = ...
ds = Dataset(...)
pps_check = FeatureLabelCorrelation().run(ds)
dup_check = DataDuplicates().run(ds)
```

Why Deepchecks here?

Deepchecks is a robust, battle-tested library for data quality. `FeatureLabelCorrelation` gives the **Predictive Power Score (PPS)** — a modern, asymmetric measure of how well a feature predicts the target (better than simple correlation for mixed data types). `DataDuplicates` catches duplicate rows early.

We explicitly pass `cat_features` to avoid Deepchecks warnings.

```python
weak_candidates = [f for f, pps in pps_check.value.items() if pps < pps_threshold]
```

Why?

Features with very low PPS are unlikely to help the model. Dropping them early reduces noise and dimensionality in the silver layer.

```python
```

```
high_signal_count = len([f for f, pps in pps_check.value.items() if pps >
0.1])

if high_signal_count > 15 and vif_threshold < 20.0:
    print("High-Density Signal Detected...")
    vif_threshold = 25.0
    pps_threshold = 0.001
```

This is the most important smart decision.

Why was this logic added?

In high-quality medical datasets (like Wisconsin Breast Cancer), many features have genuine signal. A strict VIF=10 or aggressive PPS threshold can remove too many correlated-but-useful features (e.g., multiple texture or shape measurements that are biologically related).

The rule says:
"If we detect **many strong features** (>15 with PPS > 0.1), be more conservative — raise VIF floor to 25.0 and lower PPS threshold dramatically."

This protects against **over-cleaning** in domains where multicollinearity is expected and informative (common in imaging, genomics, clinical data).

Summary for Architects

- **Strengths**:
  - Data-driven and adaptive (no hard-coded column names).
  - Prevents common MLOps pitfalls: dropping too much signal or keeping too much noise.
  - Produces auditable JSON output.
  - Special handling for medical/high-signal datasets.

- **Current Limitations** (for transparency):
  - VIF is only *suggested* in the JSON — it is **not actually computed** in this audit (your cleaning.py should use the suggested `vif_threshold`).
  - Relies only on two Deepchecks checks instead of the full `data_integrity()` suite.
  - PPS extraction assumes `pps_check.value` structure (can be brittle if Deepchecks changes).

Recommendation

This audit is solid as a **Phase 2 gate** before cleaning. It makes your pipeline more intelligent and safer for real-world datasets.

If you want, I can help you evolve it to:

- Actually compute VIF inside the audit.
- Integrate full Deepchecks `data_integrity()` suite.
- Add conflicting labels detection here (pre-clean).
- Make thresholds even more configurable via constants or config file.

Would you like a polished, production-ready version of this audit with the above improvements? Or shall we focus on how cleaning.py should consume this audit JSON?